

AI for Economic Research: Dynamic Models, Language, and Agents

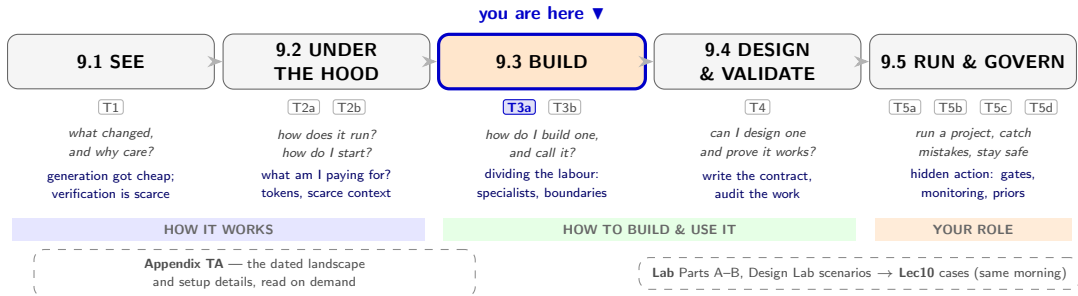
Lecture 9.3a: Build an Agent From Scratch — Eight Steps, One Canvas

Zhigang Feng

2026-09-12 15:59:24

Lecture 9 in One Map: Build an Agent

This part answers: how do I build an agent, and how do I call it?



Route: why three agents, not one → the eight steps and the canvas → tool, skill, sub-agent: where the reasoning lives.

Why Three Agents

Why the capstone needs three backgrounds — and the eight steps that build one

The Capstone Needs Three Backgrounds. The Syllabus Requires One.

The course assumes you are an economist. The capstone assumes you are also a programmer and a mathematician — and those two are exactly what it never asks for.

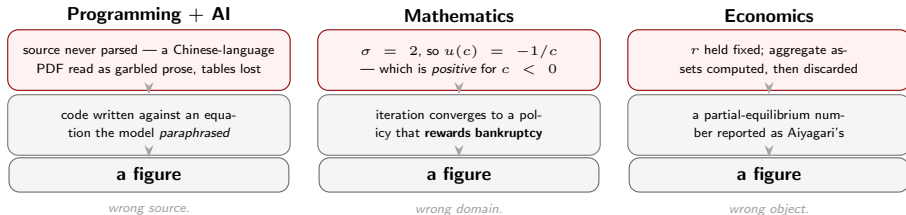
Background	What M3 “replication verified” needs	What the course assumes
Economics	an estimand or equilibrium concept, and a calibration you can defend	required — “intermediate micro and macro”
Mathematics	the domain of $u(\cdot)$, the optimality condition, the contraction, a fine enough grid	not required — recursive methods are “a plus”
Programming + AI	readable sources, a solver that runs, pinned environment, one-command reproduction	not required — “Python experience helps but is not required”

The gap is the assignment. The syllabus is not at fault — the course builds up what it needs. The problem is *sequencing*: on the day the capstone starts you are short two of the three, and the deliverables do not wait.

Answered next: what goes wrong when a background is missing — and why the output looks fine either way.

Three Error Channels — and All Three Render a Figure

A missing background does not announce itself. Each of the three produces a different error, in a different place — and all three produce a program that exits 0 and draws a clean figure.



Why three, and not one careful generalist. The three errors live in different places and are caught by different *reading*: a parse failure by counting words, a domain error by asking where a function is defined, a missing market-clearing condition by knowing what the paper's contribution was. **No single pass finds all three.**

Answered next: three people, or three job descriptions?

Hire Three Students, or Write Three Job Descriptions

The capstone allows a team of two to four with mixed skill sets, and says solo is entirely feasible. What a team supplies in people, a solo student supplies in files.

	Three students	Three agents, solo
Written at M1	a division of labour	three files in <code>.claude/agents/</code>
Fixed cost	recruiting, scheduling, trust	one afternoon of careful English
Marginal cost	their time, and yours	tokens, and <i>your</i> coordination cost (T3b)
Independent judgement	three priors, three literatures read	executorial only — all three inherit your framing (T5c)
Still entirely yours	the question, the validation, the defence	the question, the validation, the defence

A complement, not a substitute. A mathematician's job description does not give you a mathematician's judgement — it gives you a second reading by something instructed, in writing, to look only for mathematics. T5 makes the argument; the rule is: if you have a co-author who knows the math, use the co-author **and** the agent.

The Trio, Sorted by Which Background Supplies the Expertise

T5b sorts your RA team by review function. Sort the same folder by which human background supplies the expertise and a different — emptier — picture appears.

tooling-agent	math-agent	econ-agent
Programming + AI. Sources into context, models into runnable code: PDF to Markdown to $\text{\LaTeX}$, pinned environment, one-command reproduction. <i>Read, Bash, Glob, Write</i>	Mathematics. The derivation and the method: domain of $u(\cdot)$, the optimality conditions, the contraction, the grid and the tolerance. <i>Read, Grep, Glob</i>	Economics. The <i>object</i> : estimand or equilibrium concept, market clearing, calibration frequency, what the literature knows. <i>Read, Grep, Glob</i>

Now read the lab's own team again, and find the hole. code-reviewer checks the Python; domain-reviewer checks the code against the *stated* model; verifier runs it. **Nobody checks the stated model against the correct one.** The instructor key proves it: “utility() divides by $1 - \sigma$, zero in the log case” is filed under domain-reviewer — an *economist* auditing a limiting case, because no file owns that job. **[solid]**

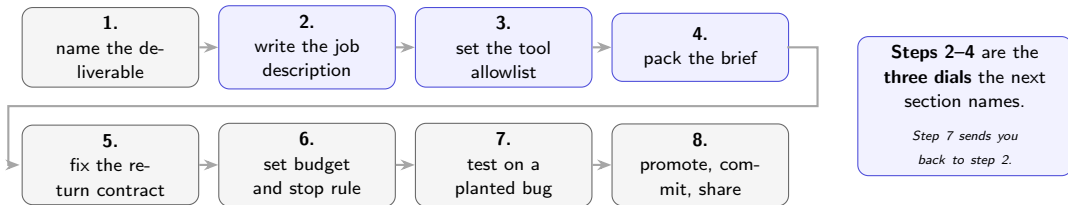
T5b sorts these files by when they load; this section sorts them by which background wrote them. Only the second question tells you what is missing — and it is the mathematics agent you are going to write.

The Eight Steps

From a job description to a file you can dispatch

Eight Steps, Eight Artifacts — You Are Building an Agent, Not a Model

T5a's six-step ladder builds a model. This ladder builds the thing that reviews it. Same discipline: every step ends in an artifact.



Eight artifacts, in order. A sentence of the form “it produces X; I will know it failed when Y”; a system prompt; a tools: line; a task string; a return format; a token ceiling and a stop rule; a 3×3 detection matrix; a committed file. **Seven of the eight produce English, not code — and a step that produced no artifact was skipped.**

Steps 1–2 — Name the Failure, Then Write the Job Description

Step 1 is one sentence: it produces X, and I will know it failed when Y. **Step 2** is that sentence expanded — the only part of an agent that is not a setting.

```
---
name: code-reviewer
description: Reviews Python numerical code for
  bugs, convergence problems, and silent failure
  modes. Use when the user asks for a code review
  or reports that results look wrong. Does NOT
  judge economics -- domain-reviewer's job.
tools: Read, Grep, Glob
model: sonnet
---
```

You are a careful reviewer of scientific Python.
This is research code, not production software:
correctness and silent failure matter far more than
style. Report, in this order:

1. Outright bugs -- wrong index, wrong sign,
wrong order of operations.
2. Silent failures -- a loop that exits on
iteration count rather than a convergence
criterion; a tolerance never checked.
3. Numerical fragility -- division without a
guard; a grid too coarse for the claim.

For each finding give the line number, quote the
line, and say what goes wrong at run time. If you
find nothing in a category, say so explicitly
rather than inventing an issue.

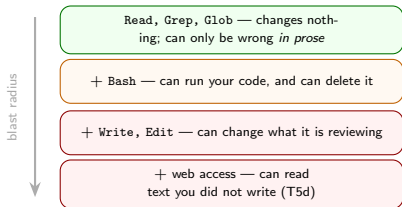
1. **Step 1 is the last clause of description:**
“Does NOT judge economics.” The failure mode
is named *before* the agent exists.
2. **Step 2 is everything below the second rule.**
That prose becomes the system argument of
every request this agent makes — T2a’s second
moving part, written by you.
3. **“say so explicitly rather than inventing an
issue”** is the most valuable sentence in the file.
An agent asked to review *will* find something.

Verbatim from this course’s own lab,

*labs/Lec09_Agent_Lab/.claude/agents/. Every slide so far
has talked about these files; this is the first one you have
seen. [solid]*

Steps 3–4 — Capability Is Blast Radius; the Brief Is the Context

The tools: line is not a convenience setting. Read forwards it is everything this agent can do to your project; read backwards it is everything it is able to learn.



Step 4 — the brief. This is what enters messages, and nothing else does:

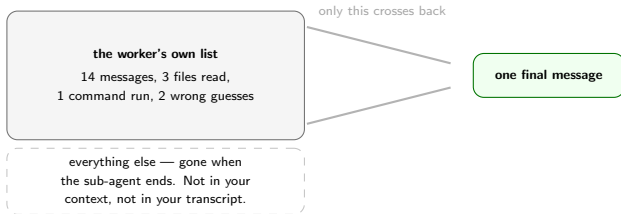
"Review `src/aiyagari.py` against `docs/model.md`. The calibration is annual. Report only findings you can tie to a line number."

What stays out: your hypothesis, your hunch about where the bug is, and the dead ends you already tried. A clean list is the entire point — T3b draws it.

The asymmetry that makes the three non-interchangeable. math-agent needs *no* shell and *no* write access — a derivation is checked by reading, and an agent that cannot execute can never confuse "it ran" with "it is right." tooling-agent is useless without Bash. Three dials, set three ways, are three different amounts of trust.

Steps 5–6 — One Message Back, and a Ceiling You Set

A sub-agent returns exactly one thing: its final message. Everything it read, ran, and reasoned about is discarded — so what you did not ask for, you will never see.



Step 5 — the return contract

- Fix the *shape*: one line per finding, giving `file:line`, the quoted text, and what breaks at run time
- Demand an explicit “nothing found in this category” — the sentence that stops an agreeable reviewer inventing one

Step 6 — budget and stop rule

- The `model`: line is a *budget* decision — a read-only reviewer on a 400-line file does not need the frontier model
- Stop rule: *report, do not fix*. A reviewer with Edit becomes an author, and you have lost your control group

Rate limits and context growth are T3b's subject. Here: you set the ceiling, per agent, in one header line.

Steps 7–8 — Test It on a Bug You Planted, Then Commit It

An untested reviewer is one you trust for no reason. Plant one defect per background, dispatch all three, and read the matrix.

Defect (or control) in the solver	tooling-agent	math-agent	econ-agent
TOL defined, never read — loop exits on MAX_ITER	✓ <i>it finished, it did not converge</i>	✓ its job — no stopping rule	—
$\sigma = 2$, so $u(c) = -1/c$, <i>positive</i> for $c < 0$	✓ <i>seen in a run</i>	✓ its job	—
r fixed; aggregate assets computed, then <i>discarded</i>	—	—	✓ its job
<i>control</i> : $EV = P @ V.T$, which is correct	silent	silent	silent

Why the planted bug is a mathematics bug. With $\sigma = 2$, $utility(c) = c^{-1}/(-1) = -1/c$: negative for feasible $c > 0$, **positive** for infeasible $c < 0$. Infeasible therefore scores *higher* — +9.49 at $c = -0.13$ against -2.00 at $c = 0.50$ — and the program prints $V[0,0] = 9.490446$, then exits 0. **[solid]**

Read the matrix, not the reports. An empty column means the description is too narrow; a full column, too broad.

Step 8: commit the file — a teammate then gets your mathematician by `git pull`.

tooling-agent at Work: A PDF Becomes Readable Context

An agent cannot read a PDF — it reads text. Turning one into the other has a cost structure an economist should recognise.

```
export LC_ALL=en_US.UTF-8    # CJK paths, spaces

# stage every not-yet-converted PDF, then call
# the converter ONCE over the whole directory
STAGE="$(mktemp -d)"
for pdf in "$INPUT_DIR"/*.pdf; do
    name="$(basename "$pdf" .pdf)"
    [ -d "$OUTPUT_DIR/$name" ] && continue
    ln -s "$(readlink -f "$pdf")" "$STAGE/"
done

mineru -p "$STAGE" -o "$OUTPUT_DIR" \
    -b vlm-auto-engine

ls "$OUTPUT_DIR" | wc -l      # files kept?
wc -w "$OUTPUT_DIR"/*/vlm/*.md # words kept?
```

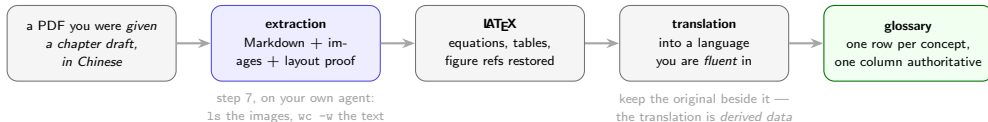
54-page econ paper, one A100-40GB	
engine startup + model load	1 m 50 s <i>fixed</i>
inference, all 54 pages	44 s <i>marginal</i>
total wall clock	2 m 59 s

Fixed cost 110 s; marginal cost 0.8 s a page. Calling the converter once *per file* re-pays the fixed cost every time: thirty PDFs become fifty-five minutes of model loading around twenty minutes of work. **One line of bash, and a returns-to-scale argument.**

Production code, NCSA Delta, via the scheduler — T5b's login-versus-compute boundary. The silent failure in the same logs: one converter's concurrency default deadlocks the engine, which stalls while the server stays bound. Nothing errors. [solid]

From Markdown to a Language You Read — and a Stable Glossary

Extraction gets you text. It does not get you a source you can cite, or a term you can use twice the same way. Those are two more artifacts.



One glossary, extended — not a new one. This lecture's Chinese script already carries a 28-row bilingual key-terms table: *harness*, *orchestrator*, *sub-agent*, *context window*, *provenance*, *CLAUDE.md*, *skill*, *hook*, *compaction*, and twenty more. Add the rows your project needs and keep one column authoritative, so the same object is not called three things in three chapters. **Style is shared; vocabulary is local.**

Why this is not busywork. A term that drifts is a variable that drifts — and the glossary is what lets *math-agent* and *econ-agent* disagree about *economics* rather than about what you meant.

The Agent Design Canvas: Eight Cells, One Worked Answer

Eight steps, one page, filled in before you write a line. Bold is the question; grey is one worked answer; the file beside it is what that answer compiles to.

1. Produces / fails when <i>a math audit; fails by passing a wrong derivation</i>	2. Job description <i>mathematics — not code, not economics</i>
3. Tools <i>Read, Grep, Glob — no shell, no write</i>	4. Brief in <i>the solver, the model note, the frequency</i>
5. Returns <i>file:line, the algebra, "correct" if correct</i>	6. Budget / stop <i>one pass; report, do not fix</i>
7. Planted bug to catch <i>$u(c) = -1/c$ is positive for $c < 0$</i>	8. Path in the repo <i>.claude/agents/math-agent.md</i>

```
---
name: math-agent
description: Checks the mathematics of a numerical economics model -- the
  domain of every function evaluated, the derivation of the optimality
  conditions, and the convergence argument. Use when results are plausible
  but the model may be misstated. Does NOT review Python style and does NOT
  judge calibration -- econ-agent's job.
tools: Read, Grep, Glob
model: sonnet
---
You check mathematics. Assume the code runs and assume the economics is
someone else's job.

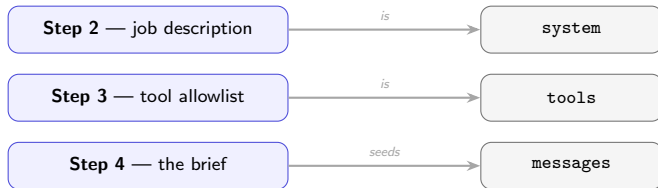
For every function the model evaluates, state its domain, check that the
code can only evaluate it inside that domain, and name the parameter value
that breaks it. Then: does the optimality condition follow from the stated
problem; is the operator a contraction, and is the stopping rule the one
that argument requires; is the grid fine enough for the claim being made?

Quote the line, give the algebra, and say plainly when a step is correct.
Never assert a result you cannot derive in the reply.
```

Then debug the English: ask it one thing it *should* handle and one it should *not*, and narrow the description until both behave (T3b). T4 hands you the blank one.

Bridge: Steps 2–4 Are the Three Dials

You have just written three job descriptions. The next section tells you what you actually did — you turned three dials on one function call, three times.



T2a's four arguments. The fourth, `model`, is step 6 — your budget dial.

What carries forward. T3b names these dials and asks how many agents you can run at once, what binds, and who decides to call them. The next section asks how a skill differs from an agent. **T3b then asks what you actually type** — and what a run with ten thousand of them consists of. Every one is now a question about a Markdown file *you* wrote.

And the milestone, plainly: M1 asks for a division of labour. Working solo, `.claude/agents/` *is* your division of labour.
Answered next: how many can run at once, and what actually stops you.

Tool, Skill, Sub-Agent

A distinction most write-ups get backwards

The Usual Definition Describes a Tool, Not a Skill

You will read that “a skill is a deterministic, hard-coded piece of software with no brain of its own.” That is an accurate definition — of something else.

The common claim

“A skill does exactly one thing. It requires exact inputs to produce exact outputs. It cannot decide when it should be used, nor fix itself on error. It just runs the code it was given.”

That paragraph is a correct description of a **tool** (a function with a schema). It is not what a skill is.

What a skill actually is

A folder with a Markdown file in it. The file is **prose written for the model to read** — a procedure, conventions, worked steps — plus optional scripts.

There is no compiled artefact and no fixed input schema. The reasoning is still the model's; the skill supplies *instructions*, not *execution*.

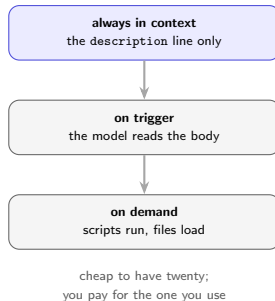
Why the confusion matters: if you believe a skill is code, you will try to write it like code — rigid parameters, defensive branching. Skills that work read like a good methods appendix written for a capable RA.

What a Skill Actually Is: Progressive Disclosure

The mechanism is a two-stage load, and it is the reason skills are cheap to keep around.

```
---
name: olg-5step
description: Manually run the local
  five-step OLG dynamic-macro demo.
  Use when explicitly invoked to
  demonstrate Model -> Equilibrium ->
  Algorithm -> Pseudo-code -> Implement.
disable-model-invocation: true
---

# OLG Five-Step Skill
Use this skill to demonstrate the five-
stage agentic dynamic-macro workflow.
## Workflow
1. Work from the demo root, the folder
   containing run.py, stages/, src/.
...
```



This repository's own skill:

`demos/M4_olg_5step_demo/.claude/skills/olg-5step/SKILL.md`

The description is load-bearing. It is the only part the model always sees, so it is what selection runs on — the same prose-driven choice as sub-agent dispatch in T3b. T5b returns to *when* skills load; this is *how* they are chosen.

Skill File Structure

```
.claude/skills/paper-review/  
  SKILL.md  
  references/  
    referee-template.md
```

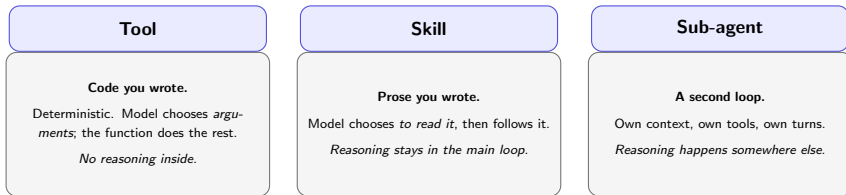
SKILL.md stores the repeatable protocol:

- what the command is called
- what steps it should follow
- what checks must pass before it reports success

Reference files store task-specific detail that would clutter the main instruction file.

Tool, Skill, Sub-Agent: Sorted by Where the Reasoning Lives

T5b sorts Skills, Agents and Rules by when they load. Sort the same objects by who does the thinking and the confusion disappears.



	Tool	Skill	Sub-agent
Made of	code	Markdown	a task description
Costs you	a tool schema	one description line	a whole second context
Fails by	raising an error	being ignored	drifting off-task

T5b's third category, Rules, is orthogonal to all three: it is always-on context, not a thing the model chooses.

disable-model-invocation: true — the **One-Line Proof**

One line in this repository's own skill settles the router question.

```
disable-model-invocation: true
```

- The flag means: *do not let the model decide to invoke this on its own* — it becomes callable by name only
- That flag is only meaningful because, by default, **the model itself** was making the call, by reading the description
- If a separate router service with a registry were dispatching, there would be nothing for a line of YAML inside one skill file to disable

T3b's dispatch argument and this section, in one sentence. Selection — of a tool, of a skill, of a sub-agent — is the model reading short English descriptions and choosing. That is why editing prose changes behaviour, and why writing good descriptions is the highest-leverage thing you do when you set up a project.

Plain English vs. Encoded Skill

Plain English

Review this paper and tell me the top problems with exact quotes.

Use when

- Exploring
- Doing a one-off task
- Still deciding the workflow

Encoded skill

`/paper-review draft_A.pdf`

Use when

- The workflow should repeat exactly
- A student team needs one standard procedure
- Validation must be consistent across versions

Exploration starts in English. Reproducibility lives in skills.

Capstone note: Track D2 asks you to deliver exactly this — a validated skill for a recurring research task.

Skills Are Saved Research Playbooks

A skill is the right home for long, repeated procedures. Keep always-loaded project context short; put domain workflows in reusable skills.

Skill	Reusable workflow	Validation hook
data-analysis	Load, explore, regress, produce tables/figures	Output schema + script rerun
review-paper	Read paper, audit argument, generate referee objections	Structured review report
weighted-survey	Compute survey-weighted rates/means	Formula check against benchmark
hpc-job	Write sbatch script, submit, parse logs	Scheduler status + parser

Portable idea: Claude may call them *skills*; Codex may package them differently. The durable object is a saved playbook with triggers, allowed tools, steps, and acceptance tests.

Placeholder — written in Phase B. The specification is in the source comment above this frame.

Bridge: From One File to Three Ways of Calling It

One agent is one Markdown file with a description that does the work.

- **Next (T3b):** how do you actually call it — by description, by name, by script — and what does a run of ten thousand consist of?